UNIVERSITY OF GHANA
FACULTY OF ENGINEERING SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING
CPEN 204: DATA STRUCTURES AND ALGORITHMS

Lab Exercise: Tree Traversals

Implementing In-order, Pre-order, Post-order, and Level-order Traversal in C++

1. Overview and Purpose

Binary trees are one of the most important non-linear data structures in computer engineering, used everywhere from file systems and databases to compilers and decision-making algorithms. Before a tree can be searched, balanced, serialized, or reasoned about, it must be traversed — visited node by node in a well-defined order.

In this lab you will implement a binary search tree (BST) in C++ and write functions to traverse it using the four standard traversal strategies: in-order, pre-order, post-order, and level-order. You will then go further: re-implementing traversals iteratively without recursion, computing structural properties of a tree in optimal time, and reconstructing an entire tree from nothing but two of its traversal sequences. The goal is not just to produce working code, but to understand what each traversal order is useful for, how its cost changes depending on how it is implemented, and why it visits nodes in the order that it does.

2. Learning Objectives

By the end of this lab, you should be able to:

- Explain the difference between depth-first traversals (in-order, pre-order, post-order) and the breadth-first traversal (level-order).
- Implement a binary search tree in C++ using nodes and pointers.
- Implement in-order, pre-order, and post-order traversal recursively.
- Implement level-order traversal iteratively using a queue.
- Predict, by hand, the output of each traversal on a given tree, and verify it against your program's output.
- Reason about when each traversal order is the appropriate one to use in practice.

3. Background: The Four Traversals

A traversal visits every node in a tree exactly once. The three depth-first orders are defined by when the node itself is visited relative to its left and right subtrees. Level-order is defined differently: it visits nodes breadth-first, level by level.

3.1 In-order (Left, Node, Right)

Visit the left subtree, then the node, then the right subtree. For a binary search tree, in-order traversal visits nodes in ascending sorted order — this is the traversal you use when you want a BST's contents printed as a sorted list.

3.2 Pre-order (Node, Left, Right)

Visit the node first, then the left subtree, then the right subtree. Pre-order is useful when you need to “copy” or serialize a tree, since a parent is always recorded before its children.

3.3 Post-order (Left, Right, Node)

Visit the left subtree, then the right subtree, then the node itself. Post-order is useful when a node depends on its children being processed first — for example, deleting a tree node by node, or evaluating an expression tree.

3.4 Level-order (Breadth-First)

Visit nodes level by level, from left to right, starting at the root. Unlike the other three, level-order is not naturally recursive; it is implemented iteratively using a queue: enqueue the root, then repeatedly dequeue a node, visit it, and enqueue its children.

4. Lab Tasks

Complete the following tasks in a single C++ program (or a small set of .cpp/.h files if you prefer to split them). All code must compile and run using a standard C++ compiler (g++, for example, with the C++11 standard or later).

Task 1: Node and Tree Structure

Define a struct or class Node representing a binary tree node, holding an integer value and pointers to its left and right children. Implement an insert() function that inserts a new value into a binary search tree according to BST ordering rules (smaller values go left, larger values go right).

Task 2: Recursive Traversals

Implement three functions, each of which takes the root of the tree and prints the values it visits, separated by spaces:

- inorder(Node* root)
- preorder(Node* root)
- postorder(Node* root)

Each of these must be implemented recursively, following the definitions in Section 3.

Task 3: Level-order Traversal

Implement a fourth function, levelorder(Node* root), that prints the tree's values in breadth-first order. This function must be implemented iteratively, using std::queue<Node*> from the C++ standard library — not recursion.

Task 4: Build, Run, and Verify

In main(), build a binary search tree by inserting the values 50, 30, 70, 20, 40, 60, 80 in that exact order. Call all four traversal functions and print clearly labelled output, for example:

```
In-order:    20 30 40 50 60 70 80
Pre-order:   50 30 20 40 70 60 80
Post-order:  20 40 30 60 80 70 50
Level-order: 50 30 70 20 40 60 80
```

Work out by hand what each traversal should produce for this tree before running your program, then confirm that your program's output matches your prediction exactly. If it does not, debug your traversal logic before moving on.

Task 5: Your Own Tree

Insert a second, different set of at least 8 distinct integer values of your choosing into a new tree. Print all four traversals for this tree as well, and by hand (on paper or in your report), draw the tree and write out what each traversal should produce before running your program — then compare your prediction to the actual output.

Task 6: Iterative Traversals — No Recursion Allowed

Recursion hides an implicit call stack from you. In this task you will make that stack explicit. Re-implement in-order and pre-order traversal — inorderIterative(Node* root) and preorderIterative(Node* root) — using an explicit std::stack<Node*> and a loop, with no recursive function calls anywhere in either function. Your output must match your Task 2 functions exactly on both the worked example and your own tree from Task 5.

This is harder than it looks for in-order: unlike pre-order, you cannot simply push a node and immediately visit it. Think carefully about when a node is pushed onto the stack versus when it is actually printed.

Task 7: Tree Metrics

Implement the following functions, each operating on the tree from Task 5:

- int height(Node* root) — the number of edges on the longest path from root to a leaf (an empty tree has height -1, a single-node tree has height 0).
- int countNodes(Node* root) — the total number of nodes in the tree.
- int countLeaves(Node* root) — the number of nodes with no children.
- bool isBalanced(Node* root) — returns true if, for every node in the tree, the heights of its left and right subtrees differ by at most 1.

Your isBalanced() must not be a naive implementation that calls height() separately at every node (which costs $O(n^2)$ in the worst case). It must compute height and check the balance condition in a single traversal, so that the overall algorithm runs in O(n) time. You will be asked to justify this in Task 9.

Task 8: Reconstructing a Tree from Traversals

Write a function Node* buildFromPreIn(vector<int>& preorder, vector<int>& inorder) that reconstructs a binary tree given only its pre-order and in-order sequences (you may assume all values are distinct). Do not use the values to re-insert into a BST — reconstruct the exact tree shape from the two sequences alone, using the standard technique: the first element of pre-order is always the root, and its position in in-order tells you the size of the left and right subtrees.

Test your function by feeding it the pre-order and in-order sequences you produced for the Task 5 tree, rebuilding the tree, and confirming (e.g. by printing its level-order traversal) that the reconstructed tree is identical to the original.

Task 9: Short-Answer Questions

Answer the following in a written section, included either as comments at the top of your source file or as a separate document in your submission. These require real explanation, not one-line answers — expect roughly half a page total per question where indicated.

1. Give a rigorous argument (not just an example) for why in-order traversal of a binary search tree always produces values in strictly ascending order. Your argument should work for a tree of arbitrary shape, not just the worked example.
2. Compare the time and space complexity of your recursive traversals (Task 2) versus your iterative, stack-based versions (Task 6). Under what real-world circumstances — think about tree shape, not just size — could the recursive version actually crash a program where the iterative version would not?

3. Pre-order plus in-order together uniquely determine a binary tree, which is what makes Task 8 possible. Explain why pre-order plus post-order do NOT always uniquely determine a binary tree — construct a small counter-example (two different trees with the same pre-order and post-order sequences) to support your answer.
4. Level-order traversal's queue can hold up to O(w) nodes at once, where w is the tree's maximum width. Compare, for a perfectly balanced tree versus a completely skewed (linked-list-shaped) tree of the same node count n, how large this queue gets in the worst case, and compare that to the maximum stack depth used by a DFS-based traversal on the same two trees.
5. Explain, at a conceptual level, how Morris in-order traversal achieves O(1) additional space by temporarily modifying the tree's own right pointers instead of using a stack or recursion. What is the trade-off it makes compared to your Task 6 solution, and why might that trade-off make it unsuitable in a multi-threaded environment?
6. For your isBalanced() function in Task 7: describe precisely why a naive version that calls a separate height() at every node degrades to $O(n^2)$ in the worst case, and explain how combining the height computation and the balance check into a single post-order pass brings this down to O(n).
7. If duplicate values are inserted into your BST's insert() function, what does your current implementation do with them? Explain one concrete way duplicates could break the reconstruction technique from Task 8, and propose a fix (either to insert() or to buildFromPreIn()).
8. Real systems rarely traverse a tree just to print it. Pick one traversal order and describe, in detail, one real system or algorithm (not already discussed in Section 3) that relies on it — for example, a compiler, a file system, a version-control tool, a database index, or a machine-learning model — and explain specifically what would go wrong if a different traversal order were used instead.

5. Requirements and Constraints

- Language: C++ (C++11 or later). Standard library containers (std::queue, std::stack, std::vector) are allowed and expected.
- The three depth-first traversals in Task 2 must be implemented recursively, not iteratively.
- The traversals in Task 6 must be implemented iteratively with an explicit std::stack, with no recursive calls.
- Level-order traversal must be implemented iteratively using a queue, not recursively.
- Task 7's isBalanced() must run in O(n) time — a naive $O(n^2)$ implementation will not receive full credit.
- Task 8 must reconstruct the tree purely from the two input sequences — it must not call insert() or otherwise re-derive the tree from BST ordering rules.
- Memory managed with raw pointers (new/delete) is acceptable for this lab; using smart pointers is optional but welcome.
- Your program must compile cleanly with no errors and should be free of warnings where reasonably possible.
- Code should be reasonably commented, especially the traversal, metrics, and reconstruction functions.

6. Deliverables and Submission

Submit the following as instructed on Google Classroom:

- Your complete C++ source file(s) (.cpp, and .h if applicable), covering Tasks 1–8.
- A written section answering all eight short-answer questions from Task 9 (this may be a separate .txt/.pdf file, or comments within your source file — follow your instructor's preference if one is stated on Classroom).

- A screenshot or copy of your program's console output for Task 4 (the worked example), Task 5 (your own tree), Task 6 (iterative traversal output), Task 7 (tree metrics), and Task 8 (reconstruction verification).